



FOR JAVA DEVELOPERS MOVING TO NESTJS

Tenant A saw tenant B.

Spring Boot to NestJS, episode 40: multi tenancy and feature flags

Request context must be scoped to the request, and rollout decisions must agree across stacks.

On Spring, a filter that never cleared its ThreadLocal leaked tenant acme into the next request on the same thread.

On Nest, the same habit as a module-level variable let two requests in flight overwrite each other. AsyncLocalStorage kept them apart.

A 20% rollout ported mechanically gave 791 of 1000 users a different answer on the two stacks. OpenFeature with one flags file gave none.

In scope

- ThreadLocal and AsyncLocalStorage
- Request scope and its cost
- Percentage rollouts across stacks
- OpenFeature with flagd

Not in scope

- Database-per-tenant routing
- Flag management UIs

flagd evaluates fractional rollouts with the same hashing in every SDK.

01 The measurements

SPRING, ThreadLocal NOT CLEARED	the next request saw acme
NEST, MODULE VARIABLE	acme read globex
NEST, AsyncLocalStorage	each saw its own
SINGLETON + REQUEST-SCOPED DEP	100 instances / 100 requests
ROLLOUT PORTED MECHANICALLY	791 of 1000 differ
OPENFEATURE + FLAGD, BOTH	0 differ

Real processes; flagd in-process from one flags.json.

02 Scope the tenant to the request

On Spring, clear the ThreadLocal in finally. On Nest, run each request in its own AsyncLocalStorage store.

TenantFilter.java

```
TenantContext.set(request.getHeader("X-Tenant"));
try {
    chain.doFilter(request, response);
} finally {
    TenantContext.clear();
}
```

tenant.middleware.ts

```
export const tenantStore = new AsyncLocalStorage<string | undefined>();
export function tenantMiddleware(req: Request, _res: Response, next: NextFunction) {
    tenantStore.run(req.header('x-tenant'), next);
}
```

module variable: acme read globex; AsyncLocalStorage: each its own

Run on a real machine, 2026-10-01, by scripts/verify-tenancy.sh in the course repository.

03 Share the evaluation contract

Evaluate the rollout with the same engine and the same file on both stacks.

flags.json

```
"new-checkout": {
  "state": "ENABLED",
  "variants": { "on": true, "off": false },
  "defaultVariant": "off",
  "targeting": { "fractional": [["on", 20], ["off", 80]] }
}
```

OpenFeature + flagd on both: Spring 202, Nest 202, 0 differ

Run on a real machine, 2026-10-01, by scripts/verify-tenancy.sh in the course repository.

Before two stacks serve the same tenants

1. Is the current tenant cleared at the end of every request?
2. Is anything request-specific stored in a module variable?
3. Which providers did request scope promote?
4. Do both stacks put the same user in the same rollout cohort?
5. Is there one source of truth for every flag?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-tenancy.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	OpenFeature https://openfeature.dev

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. OpenFeature is a Cloud Native Computing Foundation project. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.